STRICT RULES FOR AGENTS
1. Strict Adherence to the Vercel Design Language
Pixel-Perfect Minimalism: All UI elements must strictly follow the design tokens outlined in DESIGN.md. Do not introduce new colors, font families, or border radii outside of the approved system.

Typography Discipline: Use strictly Inter for standard UI text and JetBrains Mono for numbers, captions, and data points.

Fluid & Smooth UI: Transitions and state changes (hover, focus, active) must be crisp, subtle, and immediate. The application must feature high-performance client-side rendering with zero lag, especially during wallet switching, opening sliding drawers, or updating graphs.

2. Zero-Bloat Component Architecture
No Heavy External Libraries: Build custom SVG charts for the Analytics Panel to maintain complete visual control and prevent bundle bloat. Do not install heavy third-party charting libraries (like Chart.js or Recharts).

Vanilla CSS Only: Stick to Vanilla CSS utilizing the defined CSS variables. Do not install utility frameworks like Tailwind or component libraries like Material UI, as the foundational Vercel-style CSS architecture is already defined.

Reusable Primitives: Do not write one-off inline styles for buttons, inputs, or cards. Always use and extend the established reusable primitives (e.g., Button.tsx, Input.tsx, Card.tsx).

3. Strict Type Safety & Code Quality
TypeScript Enforcement: The use of the any type is strictly prohibited. All component props, context states, and local storage payloads must have explicitly defined interfaces.

Zero Compilation Errors: The build command (npm run build) must pass without a single warning or error. Code with failing types or ESLint warnings cannot be merged.

4. Modular Data & State Management
Local Storage First: All CRUD operations must read from and write to Local Storage seamlessly to allow immediate, out-of-the-box testing without backend configuration.

Backend-Ready Architecture: The data layer must be heavily modularized. Write the Local Storage logic using an interface that mimics asynchronous API calls. This ensures it can be cleanly swapped out for PostgreSQL or Supabase in the future without having to rewrite any React components.

5. Functional & Responsive Verification
Mobile-First Responsiveness: The grid layouts, navigation (slide-out drawers/sheets), and custom SVG charts must be fully fluid and perfectly usable on mobile devices.

Feature Completeness: Before marking a task complete, developers must manually verify core workflows: ensuring form submissions instantly update the global context/graphs, and validating that the CSV export formats exactly as required (Date,Description,Category,Amount,Wallet).

Implementation Plan: Vercel-Style Online Expense Tracker

An inspired, premium expense tracker built on Vercel's stark design system (from [DESIGN.md](file:///c:/Users/ashmi/Desktop/expense%20tracker/DESIGN.md)), merging the tracking power of Spendee and Expensify with developer-platform speed and minimalism.

User Review Required

> [!IMPORTANT]
> **Tech Stack Choice**: We will build this using **Vite + React + TypeScript** for maximum interactivity, type-safety, and production readiness, styled with **Vanilla CSS** matching the design token values of [DESIGN.md](file:///c:/Users/ashmi/Desktop/expense%20tracker/DESIGN.md).
>
> **Data Persistence**: To enable immediate out-of-the-box usage (for you and your friends to test without setting up servers or databases), we will implement a robust **Local Storage** database layer. It will be modularized so you can easily swap it with a PostgreSQL/Node.js or Supabase backend later.

Proposed Changes

We will construct a new React application in the workspace directory: `c:\Users\ashmi\Desktop\expense tracker`.

1. Project Initialization & Tooling
[NEW] [package.json](file:///c:/Users/ashmi/Desktop/expense%20tracker/package.json) & Configuration files
- Create a Vite React-TS project.
- Add standard configurations (Vite config, TS config, ESLint).
- Dependencies: `lucide-react` for premium icons, `canvas-confetti` for milestones (optional polish). We will build custom SVG charts to ensure perfect visual control and zero charting library bundle bloat.

2. Styling Foundation (Design System)
[NEW] [index.css](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/index.css)
- Implement CSS variables using the design tokens from [DESIGN.md](file:///c:/Users/ashmi/Desktop/expense%20tracker/DESIGN.md):
  - **Colors**: Canvas (`#ffffff`), Canvas Soft (`#fafafa`), Ink/Primary (`#171717`), Hairline (`#ebebeb`), Link Blue (`#0070f3`), gradients, etc.
  - **Typography**: Inter (open-source geometric sans substitute) and JetBrains Mono (monospaced caption/number face).
  - **Radii**: 100px pills for primary CTAs, 6px for in-app buttons/inputs, 8px for cards.
  - **Shadows**: Multi-stop stacked shadows (Level 1 to Level 5) for crisp borders and natural depth.
  - **Hero mesh gradient**: High-fidelity CSS/SVG mesh gradient representing develop/preview/ship states.

3. Core UI Components (Vercel-style Primitives)
[NEW]
[Button.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/ui/Button.tsx)
- Reusable button supporting `primary` (100px pill, ink bg), `secondary` (100px pill, white bg with hairline), `primary-sm` (pill-sm), `secondary-sm`, and `nav-cta` variants.
[NEW]
[Input.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/ui/Input.tsx)
- Premium forms inputs with hairline borders, proper focus states, and caption-mono labels.
[NEW]
[Card.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/ui/Card.tsx)
- Reusable Vercel-style container cards supporting standard marketing layouts, soft fills, and stacked elevations.

4. Application Logic & Database Layer
[NEW]
[ExpenseContext.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/context/ExpenseContext.tsx)
- Manage local state for:
  - **Transactions**: ID, Amount, Description, Category, Date, Wallet, Tags.
  - **Wallets**: Multi-wallet support (e.g., Personal, Work, Cash).
  - **Budgets**: Set and monitor category limits.
  - CRUD operations + LocalStorage synchronizer.

5. Pages & Views
[NEW]
[NavBar.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/NavBar.tsx)
- Sticky header featuring Vercel-style tabs, active indicator bar, logo, and action buttons.
[NEW]
[HeroSection.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/HeroSection.tsx)
- Full-bleed hero band featuring a massive multi-color mesh gradient backdrop, period-terminated display headline, and conversion CTAs.
[NEW]
[Dashboard.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/Dashboard.tsx)
- Grid layout displaying:
  - Key statistics cards (Balance, Income, Expenses, Budgets).
  - Add/Edit Transaction Form (in an elegant slide-out sheet/drawer).
  - Wallet Manager widget.
[NEW]
[AnalyticsPanel.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/AnalyticsPanel.tsx)
- Interactive, responsive graphs:
  - Sleek **Line Graph** mapping spending trends over time.
  - Interactive **Donut Chart** representing category breakdown with hovering tooltips.
  - **Budget Tracking Cards** showing progress bars with dynamic warning/error colors.

[NEW]
[TransactionList.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/TransactionList.tsx)
- Filterable and searchable data table featuring monochrome category icons.
- Option to trigger a custom CSV report generation & download.
[NEW]
[Footer.tsx](file:///c:/Users/ashmi/Desktop/expense%20tracker/src/components/Footer.tsx)
- Clean four-column footer with Geist Mono labels and descriptive navigation rows.

Competitor Analysis & Enhancements
- **Spendee Features**: We include visual donut charts, multi-wallet accounts, and progress indicators. *Enhancement*: High-performance client-side rendering with instant wallet switching and no lag.
- **Expensify Features**: Quick expense adding, filtering, and report exports. *Enhancement*: Clean Vercel developer-centric aesthetic, instant CSV downloads, and category-level budget locks.

Verification Plan

Automated Tests
- Validate TypeScript compilation: `npm run build`
- Validate CSS layout & responsive behavior.

Manual Verification
- **Add Transactions**: Confirm items update context and graphs.
- **CSV Export**: Trigger export, verify format matches: `Date,Description,Category,Amount,Wallet`.
- **Responsive Review**: Validate mobile overlay navigation, grid collapsing, and chart fluid resizing.